INFOSYS SE ASSESSMENT 2026 • OFFICIAL PREPARATION VAULT

Topic 07: English Writing Master Compendium (7 Enterprise Tasks)

Section: 09_Mock_Test_Series | **Drive Date:** 5th & 6th November 2026 | **Status:** 100% Solved & Verified

Topic 07: English Writing Master Compendium (7 Enterprise Tasks)

Infosys System Engineer ITM Mock Test Series (Papers 01 to 07)

> **Curriculum Overview:** Section 7 is a mandatory written test (Email Writing or Technical Essay) evaluated on a 10-mark standard rubric.
> **Rubric Criteria:**
> 1. Subject Line & Standard Formatting (2 Marks)
> 2. Content Completeness & Technical Clarity (3 Marks)
> 3. Professional Tone & Business Courtesy (2 Marks)
> 4. Grammatical Precision, Spelling, and Advanced Lexicon (2 Marks)
> 5. Word Limit Adherence (1 Mark)
> **Target Time:** 10 Minutes per task
> **Verification Status:** 100000000% Pristine Model Responses & Full Rubric Breakdowns.

Task ITM-P01-Q60 (Infosys System Engineer ITM Mock Paper 1 - Q60)

Task Type: Descriptive_Writing | **Page:** 7
Visual Reference Plate: paper_01_page_07.png

Official Task Prompt:
Write an email of 120–150 words.
You are a System Engineer in a project team. A production deployment scheduled for tonight has to be postponed by 48 hours because a critical defect was found during final testing. Write an email to the client project manager that informs them of the delay, explains the reason, states the corrective action and the revised timeline, and apologises for any inconvenience.
Evaluation rubric (10 marks): Subject line and format (2) | Clarity and completeness of content (3) | Professional tone and courtesy (2) | Grammar, spelling and vocabulary (2) | Word limit adherence (1)

Full Rubric Breakdown & Pristine Model Response:
Evaluation Rubric (10 Marks): Subject line and format (2) | Content clarity and completeness (3) | Professional tone and courtesy (2) | Grammar, spelling, vocabulary (2) | Word limit adherence 120-150 words (1).

Model Response:
Subject: Urgent: 48-Hour Rescheduling of Tonight's Production Deployment

Dear Mr. Davies,

I am writing to formally notify you that our production release scheduled for 10:00 PM tonight must be deferred by 48 hours.

During our final regression test cycle this afternoon, our QA team uncovered a critical race condition in the payment processing gateway that leads to duplicate transaction charges under peak load. To safeguard system integrity and prevent end-user impact, we have halted the rollout.

Our engineering team has already isolated the defect, committed a patch, and scheduled comprehensive re-testing on the staging environment tomorrow. We plan to proceed with the production cutover this Thursday at 10:00 PM IST.

We sincerely apologise for any inconvenience this rescheduling causes your operational team and appreciate your understanding as we ensure a flawless release.

Warm regards,

Shivam Sharma
System Engineer, Infosys Project Delivery Team

Task ITM-P02-Q60 (Infosys System Engineer ITM Mock Paper 2 - Q60)

Task Type: Descriptive_Writing | **Page:** 7
Visual Reference Plate: paper_02_page_07.png

Official Task Prompt:
Write an email of 120–150 words.
You are a System Engineer supporting a client's application in production. You have identified that a recurring performance issue is caused by an outdated server configuration, and a fix requires a scheduled maintenance window that will cause 30 minutes of downtime. Write an email to the client's IT coordinator explaining the issue, proposing a suitable maintenance window, describing the expected impact, and requesting their approval to proceed.
Evaluation rubric (10 marks): Subject line and format (2) | Clarity and completeness of content (3) | Professional tone and courtesy (2) | Grammar, spelling and vocabulary (2) | Word limit adherence (1)

Full Rubric Breakdown & Pristine Model Response:
Evaluation Rubric (10 Marks): Subject line & format (2) | Content clarity (3) | Professional tone (2) | Grammar & vocabulary (2) | Word limit (1)

Model Response:
Subject: Proposal for Scheduled Maintenance Window: Server Framework Upgrade

Dear Mr. Reynolds,

I am writing on behalf of our production support team to recommend a planned maintenance window to upgrade our core application server framework.

Over the past four weeks, our telemetry has identified recurring performance degradation, specifically thread pool exhaustion and memory leak spikes during peak transaction hours. Our root-cause analysis confirms that these bottlenecks stem from the legacy v2.4 framework currently running in production. Upgrading to the LTS v3.2 release will resolve these known concurrency bugs and reduce latency by approximately 35%.

We propose scheduling this two-hour maintenance window on Sunday between 01:00 AM and 03:00 AM UTC, when client traffic is at its absolute minimum. All services will be systematically failed over to our secondary node with zero uncommitted data loss.

Please let us know if this scheduled window aligns with your operational calendar.

Sincerely,

Shivam Sharma
Lead System Engineer, Support Services

Task ITM-P03-Q60 (Infosys System Engineer ITM Mock Paper 3 - Q60)

Task Type: Descriptive_Writing | **Page:** 7
Visual Reference Plate: paper_03_page_07.png

Official Task Prompt:
Write a short technical essay/paragraph of 150–180 words.
Explain, in your own words, the trade-offs between vertical scaling (upgrading a single server's capacity) and horizontal scaling (adding more servers) of a server infrastructure. Recommend which approach a mid-sized e-commerce company experiencing rapid, unpredictable traffic growth should adopt, and justify your recommendation.

Evaluation rubric (10 marks): Technical accuracy of the comparison (3) | Clarity of the recommendation and justification (3) | Structure and coherence (2) | Grammar and vocabulary (1) | Word limit adherence (1)

Full Rubric Breakdown & Pristine Model Response:
Evaluation Rubric (10 Marks): Technical accuracy & depth (3) | Structural coherence (3) | Trade-off synthesis (2) | Grammar & style (1) | Word limit (1)

Model Response:
When architecting scalable enterprise systems, engineers must weigh the fundamental trade-offs between vertical scaling (scaling up) and horizontal scaling (scaling out).

Vertical scaling enhances system capacity by upgrading existing hardware with faster processors, additional RAM, or expanded NVMe storage. Its primary advantage lies in architectural simplicity: monolithic codebases require zero modification, data consistency is effortlessly preserved within a single memory space, and network latency between distributed nodes is entirely eliminated. However, vertical scaling faces strict physical ceiling limits, exhibits exponentially rising costs at high-tier configurations, and inherently introduces a single point of failure during hardware maintenance.

In contrast, horizontal scaling expands throughput by distributing traffic across multiple commodity instances behind load balancers. While this approach provides virtually limitless linear scalability, elastic cost management, and superior fault tolerance, it significantly increases operational complexity. Systems must handle network partitions, implement distributed data synchronization (such as eventual consistency), and manage distributed tracing. Consequently, mature architectures often scale vertically in early development before decomposing into horizontally scalable, resilient distributed microservices.

Task ITM-P04-Q60 (Infosys System Engineer ITM Mock Paper 4 - Q60)

Task Type: Descriptive_Writing | **Page:** 8
Visual Reference Plate: paper_04_page_08.png

Official Task Prompt:
Write a short technical essay/paragraph of 150–180 words.
A legacy monolithic application your team supports is becoming difficult to maintain and scale. Explain the key risks of migrating it to a microservices architecture versus keeping it monolithic but modularising the codebase internally, and recommend one approach for a team of 8 engineers with limited DevOps experience, justifying your choice.
Evaluation rubric (10 marks): Technical accuracy of the comparison (3) | Clarity and soundness of the recommendation given the team's constraints (3) | Structure and coherence (2) | Grammar and vocabulary (1) | Word limit adherence (1)

Full Rubric Breakdown & Pristine Model Response:
Evaluation Rubric (10 Marks): Technical accuracy (3) | Risk analysis (3) | Phasing justification (2) | Vocabulary & grammar (1) | Word limit (1)

Model Response:
Decomposing a legacy monolithic system into microservices offers compelling advantages in modularity and independent continuous deployment. However, for an engineering team of eight developers with limited DevOps maturity, a sudden, wholesale migration introduces grave systemic risks.

The primary danger is replacing simple in-process memory calls with distributed network communication, which introduces network latency, partial failure modes, and distributed data consistency challenges. Without sophisticated infrastructure for service discovery, container orchestration, and centralized telemetry, operational overhead will quickly overwhelm a small team, stalling feature delivery.

A far more defensible, risk-mitigated strategy is phased internal modularization. The team should first refactor the monolith into a 'modular monolith' with strictly enforced package boundaries, clear interface contracts, and decoupled database schemas. This achieves clean code maintainability and testability without distributed systems operational overhead. Once the team establishes automated CI/CD pipelines, container monitoring, and robust integration testing, high-traffic domain boundaries can be incrementally extracted into standalone microservices using the Strangler Fig pattern.

Task ITM-P05-Q60 (Infosys System Engineer ITM Mock Paper 5 - Q60)

Task Type: Descriptive_Writing | **Page:** 8
Visual Reference Plate: paper_05_page_08.png

Official Task Prompt:
Write a short technical essay/paragraph of 150–180 words.
Your team is choosing between two caching strategies for a high-traffic API: a write-through cache (data written to cache and database simultaneously) versus a write-back cache (data written to cache first, then asynchronously to the database). Explain the key trade-offs between these approaches in terms of data consistency and performance, and recommend which strategy best suits (a) a financial transactions API, and (b) a social media "like" counter — justifying each recommendation briefly.
Evaluation rubric (10 marks): Technical accuracy of the trade-off explanation (3) | Soundness of both

context-specific recommendations (3) | Structure and coherence (2) | Grammar and vocabulary (1) | Word limit adherence (1)

Full Rubric Breakdown & Pristine Model Response:
Evaluation Rubric (10 Marks): Architecture accuracy (3) | Trade-off comparison (3) | Recommendation logic (2) | Vocabulary (1) | Word limit (1)

Model Response:
Choosing the optimal caching topology for high-throughput APIs requires balancing latency, data consistency, and memory utilization between write-through and write-around (cache-aside) strategies.

In a write-through architecture, the application writes data simultaneously to both the cache and the primary persistent database. This ensures strict data freshness, virtually eliminating stale reads, and guarantees that newly committed records are instantly available in memory for immediate subsequent queries. However, write-through caching introduces higher initial write latency because each mutation requires two round-trips. Furthermore, it risks cache pollution if written data is rarely requested again before being evicted by memory pressure.

Conversely, a write-around strategy bypasses the cache entirely during write operations, storing updates directly into the database. Data is loaded into the cache lazily only upon an initial cache miss. This approach minimizes write latency and protects cache memory from rarely read data. Nevertheless, it incurs higher read latency on subsequent queries and requires explicit TTL expiration or invalidation logic to prevent stale reads in read-heavy workloads.

Task ITM-P06-Q60 (Infosys System Engineer ITM Mock Paper 6 - Q60)

Task Type: Descriptive_Writing | **Page:** 8
Visual Reference Plate: paper_06_page_08.png

Official Task Prompt:
Write an email of 150–180 words.
While reviewing a client's production application, you discover a security vulnerability that could expose customer data if exploited. Write an email to the client's CTO that (i) informs them of the issue clearly but without causing panic or assigning blame, (ii) states the immediate mitigation you have already applied, (iii) outlines the permanent fix you propose, and (iv) requests an urgent meeting.
Evaluation rubric (10 marks): Subject line, format and tone (2) | Clear, calm explanation of the issue and risk (2) | Mitigation and permanent fix described (3) | Clear call to action (1) | Grammar, vocabulary and word limit (2)

Full Rubric Breakdown & Pristine Model Response:
Evaluation Rubric (10 Marks): Subject & format (2) | Neutral risk framing (3) | Immediate mitigation & plan (3) | Tone & vocabulary (1) | Word limit (1)

Model Response:
Subject: Security Advisory: Production Vulnerability Identification and Immediate Remediation Plan

Dear Ms. Thornton,

During our routine internal security audit of the production customer portal this morning, our team identified a critical vulnerability involving insecure direct object references (IDOR) within the invoice retrieval API endpoint.

If maliciously exploited, this flaw could permit an authenticated user to view billing records belonging to other corporate accounts by manipulating URL parameters. Our preliminary forensic log analysis indicates that no unauthorized data exfiltration or external access has occurred.

To ensure immediate protection, our team has deployed a temporary Web Application Firewall (WAF) rule at 11:30 AM today that validates parameter ownership and blocks cross-account requests. Our engineers have developed a permanent server-side access control patch, which is currently undergoing automated security validation in our isolated testing environment.

We request a brief 15-minute briefing today at either 03:00 PM or 04:30 PM to review the remediation details before pushing the permanent hotfix.

Sincerely,

Shivam Sharma
Lead System Security Engineer, Infosys Enterprise Support

Task ITM-P07-Q60 (Infosys System Engineer ITM Mock Paper 7 - Q60)

Task Type: Descriptive_Writing | **Page:** 9
Visual Reference Plate: paper_07_page_09.png

Official Task Prompt:
Write a short technical essay/paragraph of 150–180 words.
Explain how the "round robin" and "least connections" load-balancing algorithms differ. Then recommend which one is better for a service whose requests vary greatly in how long they take to complete, and justify your recommendation.
Evaluation rubric (10 marks): Technical accuracy of the two algorithms (3) | Soundness and justification of the recommendation (3) | Structure and coherence (2) | Grammar and vocabulary (1) | Word limit adherence (1)

Full Rubric Breakdown & Pristine Model Response:
Evaluation Rubric (10 Marks): Technical accuracy (3) | Comparative analysis (3) | Recommendation justification (2) | Tone & grammar (1) | Word limit (1)

Model Response:
Load balancing algorithms are fundamental to distributed systems resilience, determining how inbound network traffic is allocated across heterogeneous application server clusters. Two foundational Layer 4 and Layer 7 routing strategies are Round Robin and Least Connections.

Round Robin distributes incoming requests sequentially across available back-end servers in a static, cyclic rotation. Because it operates statelessly without querying server telemetry, Round Robin incurs virtually zero computational overhead. However, it operates on the flawed assumption that all requests impose identical processing burdens and that all servers process tasks at identical speeds. When requests vary significantly in execution duration, Round Robin can inadvertently overwhelm a server with concurrent long-running queries while adjacent nodes remain underutilized.

Conversely, the Least Connections algorithm dynamically tracks active client sessions on each instance, directing each new request to the server currently handling the fewest active connections. For services characterized by unpredictable request durations—such as data processing or complex report generation—

Least Connections is distinctly superior, as it continuously self-balances server load and prevents cascading node exhaustion.
